

```
package unit1;
/*Write a Java program that creates two boolean variables to represent whether it is
currently the weekend and whether it is raining. Then, print both values to the console.*/

public class task01 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        boolean whetherRaining=false;
        boolean weekend=true;
        System.out.println("Is whether raining?" + whetherRaining + "\nIs weekend ?" + w
eekend);
    }

}
```

```
package unit1;
/*Write a Java program that uses byte variables to store small numbers representing a
smartphone's battery level and the day of the month.*/
public class task02 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        byte batteryLevel=90;
        System.out.println("Your battery level is:" + batteryLevel);
    }

}
```

```
package unit1;
/*Write a Java program that uses short variables to store moderate-sized numbers representing
geographic elevations (above and below sea level)*/
public class task03 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        short mtEverest = 8848;
        short marianaTrench = -10994;
        System.out.println("--Geographical locations--");
        System.out.println(" Mount Everest: " + mtEverest);
        System.out.println(" Marina Trench" + marianaTrench);

    }

}
```

```
package unit1;
/*Write a Java program that uses int variables to track a player's high score and their total
daily steps walked in a fitness mini-game.*/
public class task04 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int dailySteps=4500;
        int highScore=7000;

        System.out.println("Your Steps Today: "+ dailySteps+"\nYour HighScore: "+highS
core);
    }

}
```

```
package unit1;
/*Create a class named Student with the following attributes:
â\200¢ Roll Number
â\200¢ Name
â\200¢ Department
â\200¢ CGPA
Create a constructor to initialize the values and display the student details using a method.*
/

class Student{
    int rollNumber;
    String name;
    String department;
    float cgpa;

    public Student(int rollNumber,String name,String department,float cgpa) {
        this.rollNumber=rollNumber;
        this.name=name;
        this.department=department;
        this.cgpa=cgpa;
    }
    public int printRollNumber() {
        return rollNumber;
    }
    public String printName() {
        return name;
    }
    public String printDepartment() {
        return department;
    }
    public float printCgpa() {
        return cgpa;
    }

    @Override
    public String toString() {
        return "Name is: " + name
            + "\nAge, Department and CGPA are: "
            + rollNumber + " " + department + " " + cgpa;
    }
}

public class task05 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Student s1= new Student(01,"Ram","CSE",9.0f);
        System.out.println(s1);

    }

}
```

```
package unit1;
/*Create a Student class with attributes:
Roll Number , Name , Department
Create a method displayDetails() and invoke it using an object*/

class Student1{
    int rollNumber;
    String name;
    String department;

    public Student1(int rollNumber,String name,String department) {
        this.rollNumber=rollNumber;
        this.name=name;
        this.department=department;
    }
    public void displayDetails() {
        System.out.println("--Student Details");
        System.out.println("Roll Number: "+rollNumber);
        System.out.println("Name: "+name);
        System.out.println("Department: "+department);
    }
}

public class task06 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Student1 s1=new Student1(01,"Ram","CSE");
        s1.displayDetails();
    }
}
```

```
package unit1;
/* Employee Salary
Create an Employee class with:
Employee ID, Employee Name, Salary
Create a method showSalary() and call it using an object.*/

class Employee{
    int employeeID;
    String employeeName;
    String salary;

    public Employee(int employeeID,String employeeName,String salary) {
        this.employeeID=employeeID;
        this.employeeName=employeeName;
        this.salary=salary;
    }

    public void showSalary() {
        System.out.println("--Employee Details--");
        System.out.println("Employee ID: "+employeeID);
        System.out.println("Employee Name: "+ employeeName);
        System.out.println("Salary: "+salary);
    }
}

public class task07 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Employee e1= new Employee(01,"Ram", "600000");
        e1.showSalary();
    }
}
```

```
package unit1;
/*Create an abstract class Shape with an abstract method area().
Create:
    â\200¢ Rectangle
    â\200¢ Circle
Implement the area() method in both classes and display the area*/

abstract class Shape{
    public abstract double area();
}

class Rectangle extends Shape{
    private double length;
    private double width;

    public Rectangle(double length,double width) {
        this.length=length;
        this.width=width;
    }
    @Override
    public double area() {
        return length*width;
    }
}

class Circle extends Shape{
    private double r;

    public Circle(double r) {
        this.r=r;
    }
    @Override
    public double area() {
        return Math.PI*r*r;
    }
}

public class task08 {
    public static void main(String[] args) {
        Shape rectangle1=new Rectangle(1.1,2.2);
        Shape circle1=new Circle(2.0);

        System.out.printf("Area of the Rectangle: %.2f\n", rectangle1.area());
        System.out.printf("Area of the Circle: %.2f\n", circle1.area());

    }
}
```

```
package unit1;
/*Create an abstract class Employee with an abstract method calculateSalary().
Create:
    â\200¢ Manager
    â\200¢ Developer
Implement the salary calculation and display the salary*/

//Abstract base class
abstract class Employee3 {
    private String name;
    private int id;

    // Constructor
    public Employee3(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Getters
    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }

    // Abstract method to be implemented by subclasses
    public abstract double calculateSalary();

    // Concrete method to display employee details
    public void displayEmployeeDetails() {
        System.out.println("-----");
        System.out.println("Employee ID : " + id);
        System.out.println("Employee Name : " + name);
        System.out.printf("Calculated Pay: $%.2f\n", calculateSalary());
    }
}

//Subclass Manager extending Employee
class Manager extends Employee3 {
    private double baseSalary;
    private double performanceBonus;

    public Manager(String name, int id, double baseSalary, double performanceBonus) {
        super(name, id);
        this.baseSalary = baseSalary;
        this.performanceBonus = performanceBonus;
    }

    // Implementing the abstract method
    @Override
    public double calculateSalary() {
        // Managers receive a base salary plus a performance bonus
        return baseSalary + performanceBonus;
    }
}

//Subclass Developer extending Employee
class Developer extends Employee3 {
    private double hourlyRate;
    private int hoursWorked;
```

```
public Developer(String name, int id, double hourlyRate, int hoursWorked) {
    super(name, id);
    this.hourlyRate = hourlyRate;
    this.hoursWorked = hoursWorked;
}

// Implementing the abstract method
@Override
public double calculateSalary() {
    // Developers are paid based on hourly rate and hours worked
    return hourlyRate * hoursWorked;
}
}

public class task09 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Employee3 manager = new Manager("Alice Johnson", 101, 7500.00, 1500.00);
        Employee3 developer = new Developer("Bob Smith", 102, 55.00, 160);

        // Displaying details and calculated salaries
        System.out.println("### Employee Payroll Breakdown ###");
        manager.displayEmployeeDetails();
        developer.displayEmployeeDetails();

    }

}
```

```
package unit1;
/*Create an abstract class Animal with an abstract method sound().
Create:
    â\200¢ Dog
    â\200¢ Cat
Display the sounds produced by each animal*/

//Define the abstract class Animal

abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void sound();
}

//Dog subclass extending Animal
class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Bark");
    }
}

//Cat subclass extending Animal
class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Meow");
    }
}

public class task10 {
    public static void main(String[] args) {
        // Create objects of Dog and Cat using Animal references
        Animal myDog = new Dog();
        Animal myCat = new Cat();

        // Display the sounds produced by each animal
        System.out.print("Dog says: ");
        myDog.sound();

        System.out.print("Cat says: ");
        myCat.sound();
    }
}
```

```
package unit1;

/*Create a class BankAccount with private variables:
â\200¢ accountNumber
â\200¢ balance
Create getters and setters to access the data.*/

public class task11 {
    // Private variables (Data Hiding)
    private String accountNumber;
    private double balance;

    // Optional: Constructor to initialize the object
    public task11(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    // Getter for accountNumber
    public String getAccountNumber() {
        return accountNumber;
    }

    // Setter for accountNumber
    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    // Getter for balance
    public double getBalance() {
        return balance;
    }

    // Setter for balance
    public void setBalance(double balance) {
        // You can add validation logic here (e.g., balance cannot be negative)
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Error: Balance cannot be negative.");
        }
    }
}
```

```
package unit1;
/*Create a class Employee with private variables:
â\200¢ employeeId
â\200¢ employeeName
â\200¢ salary
Use getter and setter methods to display the details.*/

class Employee4{
    private String employeeID;
    private String employeeName;
    private double salary;

    public Employee4(String employeeID,String employeeName,double salary) {
        this.employeeID=employeeID;
        this.employeeName=employeeName;
        this.salary=salary;
    }
    public String getEmployeeID() {
        return employeeID;
    }
    public String getEmployeeName() {
        return employeeName;
    }
    public double getSalary() {
        return salary;
    }

    public void setEmployeeID(String employeeID) {
        this.employeeID=employeeID;
    }
    public void setEmployeeName(String employeeName) {
        this.employeeName=employeeName;
    }
    public void setSalary(double salary) {
        this.salary=salary;
    }

}

public class task12 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Employee4 e1=new Employee4("01","Ramu",60000);
        System.out.println("Employee ID: "+e1.employeeID+" Employee Name: "+e1.employeeName+" Employee Salary: "+e1.salary);
        e1.setSalary(80000);
        System.out.println("Employee ID: "+e1.employeeID+" Employee Name: "+e1.employeeName+" Employee Salary: "+e1.salary);

    }

}}
```

```
package unit1;
/*Create:
  â\200¢ Parent Class: Vehicle
  â\200¢ Child Class: Bike
Methods:
  â\200¢ start()
  â\200¢ ride()
Display both methods using inheritance.*/

abstract class Vehicle{
    void start() {
        System.out.println("Starting.....");
    }
    abstract void ride();
}
class Bike extends Vehicle{
    @Override
    void ride() {
        System.out.println("Riding.....");
    }
}
public class task13 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Vehicle v1=new Bike();
        v1.start();
        v1.ride();

    }

}
```

```
package unit1;
/*Create an abstract class Vehicle with:
Abstract Method
start()
Subclasses
â\200¢ Car
â\200¢ Bike
Display how each vehicle starts.*/

abstract class Vehicle1 {
    // Abstract method (has no body)
    abstract void start();
}

// Subclass Car extending Vehicle
class Car extends Vehicle1 {
    @Override
    void start() {
        System.out.println("The car starts with a push-button ignition or turn-key.");
    }
}

// Subclass Bike extending Vehicle
class Bike1 extends Vehicle1 {
    @Override
    void start() {
        System.out.println("The bike starts with a kick-start or an electric button.");
    }
}

public class task14 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Vehicle1 myCar = new Car();
        Vehicle1 myBike = new Bike1();

        // Display how each vehicle starts
        myCar.start();
        myBike.start();
    }
}
```

```
package unit1;

/*Create a class Book with:
â\200¢ Book ID
â\200¢ Title
â\200¢ Author
Create an object and display the book details.*/

class Book {
    // Attributes (Instance Variables)
    private int bookId;
    private String title;
    private String author;

    // Parameterized Constructor to initialize the attributes
    public Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    // Method to display the book details
    public void displayDetails() {
        System.out.println("--- Book Details ---");
        System.out.println("Book ID : " + bookId);
        System.out.println("Title   : " + title);
        System.out.println("Author  : " + author);
    }
}

public class task15 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Book myBook = new Book(101, "Effective Java", "Joshua Bloch");

        // Display the book details
        myBook.displayDetails();
    }
}
```

```
package unit1;
/*Create an object for an Employee class and display:
â\200¢ Employee ID
â\200¢ Employee Name
â\200¢ Salary*/

class Employee5 {
    // Instance variables (Attributes)
    private int employeeId;
    private String employeeName;
    private double salary;

    // Parameterized constructor to initialize the variables
    public Employee5(int employeeId, String employeeName, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.salary = salary;
    }

    // Method to display the employee details
    public void displayEmployeeInfo() {
        System.out.println("--- Employee Details ---");
        System.out.println("Employee ID : " + employeeId);
        System.out.println("Employee Name : " + employeeName);
        System.out.println("Salary : $" + salary);
    }
}

public class task16 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Employee5 emp = new Employee5(101, "Alice Smith", 75000.50);

        // Display the details of the created object
        emp.displayEmployeeInfo();
    }
}
```

```
package unit1;
/*Create methods:
â\200¢ add()
â\200¢ subtract()
â\200¢ multiply()
Display the results using an object.*/

class Calculator {
    // Methods returning the results
    public double add(double a, double b) {
        return a + b;
    }

    public double subtract(double a, double b) {
        return a - b;
    }

    public double multiply(double a, double b) {
        return a * b;
    }
}

public class task17 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Calculator calc = new Calculator();

        // Define input numbers
        double num1 = 20.0;
        double num2 = 5.0;

        // Display the results using the object
        System.out.println("Addition: " + calc.add(num1, num2));
        System.out.println("Subtraction: " + calc.subtract(num1, num2));
        System.out.println("Multiplication: " + calc.multiply(num1, num2));

    }
}
```

```
package unit1;
```

```
/*Write a Java Program to demonstrate all four access specifiers using two classes in two different*/
```

```
// Helper class within the same package to hold different access levels
```

```
class DataHolder {  
    // 1. Public: Accessible from anywhere  
    public String publicVar = "Public Variable: Accessible everywhere.";   
  
    // 2. Protected: Accessible within the same package and by subclasses  
    protected String protectedVar = "Protected Variable: Accessible in same package.";   
  
    // 3. Default (No keyword): Accessible only within the same package  
    String defaultVar = "Default Variable: Accessible within the package.";   
  
    // 4. Private: Accessible ONLY inside this class  
    private String privateVar = "Private Variable: Restrictive, inside DataHolder only.";   
  
    // Public method to show all variables inside their own class  
    public void displayAll() {  
        System.out.println("--- Inside DataHolder Class ---");  
        System.out.println(publicVar);  
        System.out.println(protectedVar);  
        System.out.println(defaultVar);  
        System.out.println(privateVar); // Fully accessible here  
    }  
  
    // Getter method to allow controlled access to the private variable outside  
    public String getPrivateVar() {  
        return privateVar;  
    }  
}
```

```
public class task18 {  
  
    public static void main(String[] args) {  
        // Instantiate the helper class  
        DataHolder holder = new DataHolder();  
  
        // Display access from within the helper class itself  
        holder.displayAll();  
        System.out.println();  
  
        // Demonstrate access from a separate class (task18) in the same package  
        System.out.println("--- Accessing from task18 Class ---");  
  
        // Accessing public, protected, and default variables directly  
        System.out.println(holder.publicVar);  
        System.out.println(holder.protectedVar);  
        System.out.println(holder.defaultVar);  
  
        // CRITICAL: Un-commenting the line below will throw a compile-time error  
        // System.out.println(holder.privateVar);  
  
        // Accessing the private variable safely using a public getter method  
        System.out.println("Indirectly: " + holder.getPrivateVar());  
    }  
}
```

```
package unit1;
/*Create a class Student with:
â\200¢ Roll Number
â\200¢ Name
â\200¢ Static variable college
Create two student objects and display their details.*/
class Student6 {
    // Instance variables (unique to each object)
    int rollNo;
    String name;

    // Static variable (shared among all objects)
    static String college = "RIT";

    // Constructor to initialize instance variables
    public Student6(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    // Method to display student details
    public void displayDetails() {
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("College: " + college);
        System.out.println("-----");
    }
}

public class task19 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Student6 student1 = new Student6(101, "Alice");
        Student6 student2 = new Student6(102, "Bob");

        // Displaying details of both students
        System.out.println("--- Student 1 Details ---");
        student1.displayDetails();

        System.out.println("--- Student 2 Details ---");
        student2.displayDetails();

    }
}
```

```
package unit1;
/*Write a Java program to implement a Mobile Store using a default constructor. Create a Mobile class
with the attributes brand and price. The default constructor should display a welcome message
when
a mobile object is created. Display the mobile details after object creation.*/
class Mobile {
    // Attributes
    String brand;
    double price;

    // Default constructor (no-argument constructor)
    Mobile() {
        System.out.println("Welcome to the Mobile Store!");
        brand = "Samsung";
        price = 75000.0;
    }

    // Method to display mobile details
    void displayDetails() {
        System.out.println("Mobile Brand: " + brand);
        System.out.println("Mobile Price: â\2021" + price);
    }
}

public class task20 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Mobile myMobile = new Mobile();

        System.out.println("\nDisplaying Mobile Details After Creation:");
        myMobile.displayDetails();
    }
}
```

```
package unit1;

/*Write a Java program to implement a Mobile Store using a parameterized constructor. Create
a Mobile class with the attributes brand and price. Accept the values through the constructor
and
display the details of two different mobile objects.*/

class Mobile1 {
    String brand;
    double price;

    // Parameterized constructor to initialize brand and price
    Mobile1(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    // Method to display mobile details
    void displayDetails() {
        System.out.println("Brand: " + brand);
        System.out.println("Price: Rs " + price);
        System.out.println("-----");
    }
}

public class task21 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("=== Welcome to the Mobile Store ===");

        // Creating two different mobile objects using the parameterized constructor
        Mobile1 mobile1 = new Mobile1("Apple", 999.99);
        Mobile1 mobile2 = new Mobile1("Samsung", 799.50);

        // Displaying the details of both mobile objects
        mobile1.displayDetails();
        mobile2.displayDetails();
    }
}
```

```
package unit1;
/* Write a Java program to implement a user-defined copy constructor in a Mobile Store applica
tion.
Create a mobile object with a brand and price, then create another object by copying the first
object
using the copy constructor. Display the details of both objects.*/

class Mobile2 {
    private String brand;
    private double price;

    // Parameterized constructor to initialize the first mobile object
    public Mobile2(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    // User-defined copy constructor
    // It accepts an object of the same class as a parameter
    public Mobile2(Mobile2 otherMobile) {
        this.brand = otherMobile.brand; // Copying the brand
        this.price = otherMobile.price; // Copying the price
    }

    // Method to display mobile details
    public void displayDetails(String objectLabel) {
        System.out.println(objectLabel + " -> Brand: " + brand + ", Price: $" + price);
    }
}

public class task22 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("--- Mobile Store Application --- \n");

        // 1. Create the original mobile object using the parameterized constructor
        Mobile2 originalMobile = new Mobile2("Apple iPhone 15", 799.99);

        // 2. Create a duplicate object by passing the original object into the copy construct
or
        Mobile2 copiedMobile = new Mobile2(originalMobile);

        // 3. Display the details of both objects to verify they match
        originalMobile.displayDetails("Original Mobile");
        copiedMobile.displayDetails("Copied Mobile ");
    }
}
```

```
package unit1;
/*Write a Java program to demonstrate a no-argument constructor using a Mobile Store application.
Initialize the mobile details inside the constructor and display the details using a member function.*/

class Mobile3 {
    // Instance variables (Mobile details)
    String brand;
    String model;
    double price;
    int ramSize; // in GB

    // No-argument constructor to initialize mobile details
    Mobile3() {
        brand = "Samsung";
        model = "Galaxy S24";
        price = 79999.00;
        ramSize = 8;
    }

    // Member function to display mobile details
    void displayDetails() {
        System.out.println("--- Mobile Store Item Details ---");
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Price: â\2021" + price);
        System.out.println("RAM: " + ramSize + " GB");
    }
}

public class task23 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Mobile3 myMobile = new Mobile3();

        // Calling the member function to display the initialized details
        myMobile.displayDetails();
    }
}
```

```
package unit1;

/*Write a Java program to demonstrate constructor overloading in a Mobile Store
application. Create a Mobile class with three constructors:
â\227\217 A default constructor
â\227\217 A constructor that accepts only the mobile brand
â\227\217 A constructor that accepts both the mobile brand and price
Create objects using all three constructors and display their details.*/

class Mobile4 {
    // Instance variables
    private String brand;
    private double price;

    // 1. Default constructor
    public Mobile4() {
        this.brand = "Unknown Brand";
        this.price = 0.0;
    }

    // 2. Constructor that accepts only the mobile brand
    public Mobile4(String brand) {
        this.brand = brand;
        this.price = 0.0; // Default price when not specified
    }

    // 3. Constructor that accepts both the mobile brand and price
    public Mobile4(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    // Method to display the details of the mobile object
    public void displayDetails() {
        System.out.println("Mobile Brand: " + brand + " | Price: $" + price);
    }
}

public class task24 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("--- Welcome to the Mobile Store --- \n");

        // Creating an object using the default constructor
        Mobile4 mobile1 = new Mobile4();

        // Creating an object using the brand-only constructor
        Mobile4 mobile2 = new Mobile4("Apple iPhone");

        // Creating an object using the brand and price constructor
        Mobile4 mobile3 = new Mobile4("Samsung Galaxy", 1199.99);

        // Displaying details of all created objects
        System.out.println("Object 1 Details (Default):");
        mobile1.displayDetails();

        System.out.println("\nObject 2 Details (Brand Only):");
        mobile2.displayDetails();

        System.out.println("\nObject 3 Details (Brand and Price):");
        mobile3.displayDetails();
    }
}
```

```
}
```

```
}
```

```
package unit1;
```

```
/*Write a Java program that uses long variables to store massive numbers representing  
galactic distances and global web searches.*/
```

```
public class task25 {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        long distanceToAndromedaKm = 2400000000000000000L;
```

```
// Global web searches: Estimated total Google searches per year worldwide
```

```
long annualGlobalWebSearches = 20000000000000L;
```

```
// Print the values with descriptive text
```

```
System.out.println("--- Galactic Distances ---");
```

```
System.out.println("Distance to the Andromeda Galaxy: " + distanceToAndromedaKm + " km  
");
```

```
System.out.println("\n--- Global Web Metrics ---");
```

```
System.out.println("Estimated annual global web searches: " + annualGlobalWebSearches)
```

```
;
```

```
}
```

```
}
```